

12th. Federation of European Simulation Societies Conference, (Eurosime 2026)

A Task-Oriented Executable Modeling Framework for Smart Chemical Laboratory

Tao Yu^a, Chun Zhao^{b,*}

^aBeijing Information Science & Technology University, Qinghe Xiaoying East Road 12, Haidian District, Beijing 100192, China

^bBeijing Information Science & Technology University, Qinghe Xiaoying East Road 12, Haidian District, Beijing 100192, China

Abstract

Experimental execution in a smart chemical laboratory is not merely the sequential execution of device commands, but the dynamic evolution of the laboratory system as a whole under a specific task context. Therefore, the dynamic changes in laboratory resources, process data, workflow progression, and execution constraints during execution need to be explicitly represented. However, existing automation modeling approaches mainly focus on invocation logic, workflow structure, or static system structure modeling. They fail to organize the key elements involved in the experimental execution process into a unified structure of execution semantics, making it difficult to explicitly represent the overall dynamic evolution of the laboratory system during task execution. Based on this view, this paper proposes a task-oriented executable modeling framework for smart chemical laboratories. The framework integrates static resources, dynamic workflows, and execution constraints to provide a unified semantic basis for the experimental execution process. In this framework, a workflow is regarded as an execution slice formed by the laboratory system under a specific task context. A workflow node is defined as a process-level abstract mapping of a device execution capability. Each node carries a single execution capability and describes its execution semantics through device attribute data, lifecycle states, and interface relationships. Finally, based on a prototype system, this paper uses a reactor heating experiment as an example to illustrate the instantiated architecture and concrete implementation of the framework. The case results show that explicit modeling of the core elements involved in execution can establish consistent semantic associations among device abstraction, workflow execution, and constraint supervision. These associations form a unified semantic structure, thereby enabling the explicit representation of the dynamic evolution of a smart chemical laboratory system during task execution.

© 2025 The Authors. Check in the contract: Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 22nd International Multidisciplinary Modeling & Simulation Multiconference.

Keywords: Experimental task execution; Execution semantics; Workflow modeling; Workflow-based execution; Smart laboratory; Laboratory automation

1. Introduction

The development of smart chemical laboratories is driving laboratory automation from isolated device control toward integrated task execution [1]. Against this background, research on laboratory task execution should no longer

* Corresponding author. Tel.: +86-13311296283.

E-mail address: zhaochun@bistu.edu.cn (Chun Zhao).

1877-0509 © 2025 The Authors. Check in the contract: Published by Elsevier B.V.

This is an open access article under the CC BY-NC-ND license (<http://creativecommons.org/licenses/by-nc-nd/4.0/>)

Peer-review under responsibility of the scientific committee of the 22nd International Multidisciplinary Modeling & Simulation Multiconference.

be limited to discrete sequences of device control commands, but should instead focus on the experimental task execution process itself. Experimental task execution is not the sequential invocation of device commands, but a dynamic evolution process formed by laboratory device resources, process data, workflow progression, and execution constraints under a specific task context. Therefore, what needs to be explicitly represented is not only the invocation order of device operations, but also the dynamic changes and coordination relationships among these elements during execution [2].

However, existing laboratory automation approaches struggle to explicitly represent execution elements mentioned above. Many systems implement experimental tasks through scripts, application programming interfaces, or manually constructed orchestration logic [3]. Although such approaches enable functional automation and achieve software-hardware decoupling to a certain extent, they mainly focus on command invocation, device connectivity, and runtime control. Execution semantics such as device state transitions, data propagation, synchronization control, and constraint handling are often scattered across invocation logic, interface definitions, and runtime code. As a result, even though experimental tasks can be executed, the key elements involved in the process are not organized into a unified structure of execution semantics. This makes it difficult to explicitly represent the organizational structure and state evolution of the laboratory system at the model level during task execution [2].

Workflow-based modeling approaches can describe experimental procedures in a structured manner, yet such structured representation alone is insufficient to characterize the key execution elements in experimental task execution [4]. Traditional workflows mainly focus on step sequences and control dependencies, and can describe how an experimental task proceeds. However, such representations can hardly naturally capture the device capabilities involved in the task, the transmission paths of process data, and the constraint boundaries of execution [5]. Therefore, a workflow should not be regarded merely as a sequence of steps or device invocations. It should further represent the dynamic organizational structure formed by the laboratory system during task execution. Such a dynamic structure is neither a static replica of the complete laboratory system nor a simple sequence of device invocations. Instead, it consists of the resource subset relevant to the current task and the temporary relationships among these resources [2]. Specifically, task execution activates the device capabilities and connections related to the current task, and introduces the corresponding constraint boundaries. These elements form temporary temporal relations, control relations, and data transmission relations around the task objective during execution. Based on this understanding, this paper defines a workflow as an execution slice formed by the laboratory system under a specific task context, which represents the dynamic organization of activated resources and their relationships during task execution.

From this perspective, a workflow node is no longer a simple encapsulation of device commands or API calls. Instead, it is a process-level abstract mapping of device capabilities. Each node corresponds to one activated device capability and is associated with corresponding device attribute data, lifecycle states, and interaction relationships. In this way, the workflow is extended from a simple process description to a system organization representation under a task context, thereby providing a basis for the explicit modeling of experimental task execution.

Based on the above understandings, this paper proposes an executable modeling framework for smart laboratories oriented to experimental task execution. The framework represents the laboratory system from three dimensions, namely static resources, dynamic workflows, and execution constraints, and uses their coordination to characterize the dynamic evolution of the laboratory system during experimental task execution. The static resource layer describes laboratory device resources and represents device capabilities, device state machines, and device connections. The dynamic workflow layer organizes the device capabilities activated in the current task into a task-specific execution slice, which describes workflow progression, state evolution, and data transfer during experimental task execution. The execution constraint layer specifies the constraint boundaries at both the laboratory scope and task level, and provides continuous supervision over the execution process. Through this organization, the framework can explicitly represent the dynamic interrelations among resources, workflows, and constraints during experimental task execution.

The main contributions of this paper are as follows:

- It proposes a task-oriented executable modeling framework for smart laboratories, which defines a workflow as an execution slice formed by the laboratory system under a specific task context.
- The framework represents a workflow node as a process-level abstract mapping of a device capability and integrates resource representation, workflow organization, and constraint supervision into a unified model system to explicitly characterize the dynamic evolution of the laboratory system during experimental task execution.

The rest of this paper is organized as follows. Section 2 reviews related work on laboratory automation, workflow-based execution, and model-based representation of laboratory systems. Section 3 elaborates in detail on the methodology of the three-layer framework for the task-oriented executable modeling of smart laboratories. Section 4 conducts a case study to verify the effectiveness of the proposed framework. In Section 5, this work is summarized by concluding the main findings and proposing future research directions.

2. Related Work

Existing research on smart chemical laboratory systems can be examined from the perspective of how experimental task execution is represented. From this perspective, existing studies can be broadly grouped into three categories: script- and API-invocation-based automation methods, workflow-based process modeling methods, and model-based system representation methods [6, 7].

The first category of research mainly focuses on script-driven device automation and adapter-based API orchestration [7]. In such approaches, device heterogeneity is typically handled via interface encapsulation, and task execution is implemented through scripts, templates, or orchestration logic [3]. These methods are effective for instruction scheduling, device integration, and engineering implementation [6, 8, 9]. However, their primary focus is on operation invocation, device connection, and runtime control, rather than the explicit representation of the execution process [10]. Although device states, synchronization logic, data transmission, and constraints may all exist in the runtime system, such execution semantics are usually scattered across code, interfaces, and control logic. As a result, even though task execution can be realized, its key execution elements remain confined within the software implementation and are not abstracted into a unified model representation.

The second category of research employs workflows or process models to represent experimental procedures. Such methods facilitate the description of control flow and step dependencies [4, 11]. However, most workflow approaches still focus primarily on describing process sequences and control logic [12], where nodes are usually interpreted as process actions rather than execution abstractions mapped from device capabilities and supported by device models. Therefore, these methods are more suitable for representing the process progression of tasks, but insufficient to explicitly represent the activated resources and their relationships during experimental task execution. More specifically, they cannot naturally express the device capabilities involved in the current task, the evolution of device states, the transmission paths of process data, the coordination relationships among resources, and the constraint boundaries of execution. Thus, although workflow abstraction provides an important foundation for experimental task modeling, traditional workflow representations alone are still inadequate to fully characterize the experimental task execution process in smart laboratories.

The third category of research includes general model-based approaches and MBSE-oriented system representation methods. Compared with implementation-centered approaches, such approaches attach greater importance to the explicit representation of system structures and behavioral relationships [13, 14], which provides a foundation for the systematic description of laboratory systems instead of being limited to ad hoc execution logic [15, 16]. However, these methods mainly focus on modeling system structure and general behavior, and fail to clarify how to organize experimental task-related resources, workflows, and constraints within a unified model framework [2]. In other words, although such methods deliver system-level representation capabilities, they have not explicitly illustrated how to construct task-specific execution representations of activated resources and their relationships via workflows under specific task contexts.

Overall, although the above studies can describe device states, workflow progression, or system structure to varying degrees, they have not established a unified representation of the key execution elements involved in experimental task execution. Based on this, this paper proposes an executable modeling framework for experimental task execution. The framework defines a workflow as an execution slice formed by the laboratory system under a task context, and integrates resources, workflows, and constraints into a unified model system to explicitly represent the dynamic evolution of experimental task execution.

3. Methodology

Experimental task execution is not merely the sequential invocation of device commands, but the overall dynamic evolution of the laboratory system under a specific task context [17]. Therefore, the explicit modeling of experimental task execution must cover three core elements, resources that persist and evolve during operation, workflow organization tailored to specific tasks, and constraints that ensure all execution activities remain within permissible boundaries [18]. Based on this understanding, this paper constructs a unified modeling framework that characterizes experimental task execution from three perspectives, namely resource representation, workflow abstraction, and constraint supervision. The following subsections describe the framework in detail.

3.1. System Abstraction

The laboratory model is abstracted as

$$\mathcal{L} = \langle \mathcal{R}, \mathcal{F}, \mathcal{C}, \Sigma \rangle \quad (1)$$

where $\mathcal{R}$ denotes the static resource layer, $\mathcal{F}$ denotes the dynamic workflow layer, $\mathcal{C}$ denotes the constraint layer, and Σ denotes the dictionary of interaction signals.

The components are defined as follows.

Static Resource Layer ($\mathcal{R}$): This layer models the available device assets in the laboratory, the intrinsic operational capabilities of devices, and the rules of state evolution during operation. It provides the structural basis for laboratory task execution.

Dynamic Workflow Layer ($\mathcal{F}$): This layer models the device capabilities activated during task execution and their organizational relationships. It represents an experimental procedure as an execution slice formed by the laboratory system under a specific task context. In this slice, workflow nodes correspond to process-level mappings of activated device capabilities, while workflow relations characterize the temporary temporal, control, and data relationships formed among activated resources.

Constraint Layer ($\mathcal{C}$): This layer models execution boundary conditions. It describes laboratory-wide boundary constraints and supplementary constraints introduced during task execution, and monitors whether the current execution remains within the admissible range.

Dictionary of interaction signals (Σ): This component defines the signal events used for interaction within the modeling framework.

Under this abstraction, the three layers correspond to three coordinated semantic roles. The resource layer provides the executable basis of the laboratory, the workflow layer represents the execution slice of the system under a specific task, and the constraint layer supervises the operational boundaries of the system.

3.2. Static Resource Layer

The static resource layer $\mathcal{R}$ models the available device assets in the laboratory and their connections. It describes the static resource structure of the laboratory and provides a unified and executable semantic basis for the other layers.

The core modeling principle of this layer is to separate device capabilities from device states. Specifically, each physical device is abstracted as a device object model composed of a device capability model and a device state machine. The capability model describes the static information and capability semantics of the device, while the state machine describes execution interactions and state transitions.

Formally, the static resource layer is defined as

$$\mathcal{R} = \langle V_D, G_C \rangle \quad (2)$$

where $V_D = \{D_1, D_2, \dots, D_n\}$ is the set of device object models, and G_C is the organizational structure of device resources.

Each device object model $D \in V_D$ is formally defined as $D = \langle C_D, FSM_D \rangle$, where C_D is the device capability model and FSM_D is the device state machine.

The device capability model is defined as

$$C_D = \langle Att, \Phi, \Gamma, P_D, \Lambda_D \rangle \quad (3)$$

where Att denotes the device attribute space, including continuous and discrete attributes; Φ denotes the set of operational capabilities exposed by the device; Γ denotes the adapter contract, which specifies command templates, interaction events, and feedback signals between the adapter and the device object model; P_D denotes the set of attribute-data interaction ports; and Λ_D denotes the device-intrinsic constraints, namely the set of device-level operational constraints that remain active independently of the task context.

Specifically, each element in Λ_D is defined as a device-local condition $\lambda = \langle att, \odot, v \rangle$, where att is a device attribute, $\odot$ is a comparison operator, and v is a constraint boundary or value range. During operation, the device state machine continuously monitors device attributes and states according to the rules specified in Λ_D . Once a constraint violation is detected, the state machine immediately enters the corresponding abnormal state.

The adapter contract Γ is logically associated with an implementation component called the adapter. The adapter controls interactions with the physical device and reports execution status to the device object model. In practice, the adapter registers its command templates and event signals with the model, while the capability model provides a unified semantic view of this information.

In this design, the capability model serves as the static semantic basis of the device. It does not actively execute operations. Instead, it provides a structured semantic description of device attributes, executable capabilities, and device-intrinsic constraints. Device attributes can be obtained directly from Att , without being relayed through the state machine.

The dynamic behavior of the device is controlled by the device state machine, which is defined as

$$FSM_D = \langle Q, I, M, T, q_0 \rangle \quad (4)$$

where $Q = Q_D^{cmd} \times Q_D^{op}$ denotes a composite state space consisting of command lifecycle states and device operation states; I denotes the interface set; M denotes the set of internal state-machine actions; T denotes the set of state-transition rules; and q_0 denotes the initial state.

Dividing the state space into command lifecycle states and device operation states reflects the dual nature of device execution. The command lifecycle state describes the execution progress of an operation command, while the device operation state reflects the actual physical operating condition of the device.

The interface set I defines the interaction boundary of the state machine. Specifically, i_{cmd}^{in} receives device control commands, i_{adp}^{in} receives feedback events from the adapter, i_{adp}^{out} sends operation commands to the adapter, and i_{stat}^{out} publishes device states externally. Interfaces serve only as communication endpoints and do not carry execution logic.

The internal action set M contains a minimal set of action primitives, such as sending control commands to the adapter or emitting signals through interfaces. These actions are triggered during state transitions or upon state entry.

Each state-transition rule $t \in T$ is defined as

$$t = \langle q_s, i, \sigma, q_t, A_t \rangle \quad (5)$$

where q_s and q_t denote the source state and target state, respectively; i denotes the interface that receives the triggering signal; σ denotes the triggering signal; and $A_t \subseteq M$ denotes the set of actions executed during the transition. State transitions in the device state machine are jointly determined by the current state, the input interface, and the received signal. Each state may also define corresponding entry actions.

The system connection graph is defined as

$$G_C = \langle P, E_C \rangle \quad (6)$$

where $P = \bigcup_{D \in V_D} P_D$ denotes the global set of device ports in the laboratory, and $E_C \subseteq P \times P$ denotes the set of port-to-port connections. This graph describes attribute-level data exchange relationships among device assets.

During experimental task execution, the static resource layer serves as the operational basis of the laboratory system. Workflow nodes interact with device object models through state-machine interfaces. After receiving a control signal, the state machine performs a state transition and may send operation commands to the adapter during the

transition. It then receives feedback events from the adapter and publishes device states through its interfaces. In addition, the state machine continuously monitors the operating condition of the device according to Λ_D . Once the device violates a constraint boundary, the state machine immediately enters the corresponding abnormal state. Through this mechanism, physical devices are transformed into controllable, state-aware, and executable digital object models.

3.3. Dynamic Workflow Layer

The dynamic workflow layer $\mathcal{F}$ explicitly models the process of laboratory task execution. It focuses on the device capabilities activated during execution, the organizational relationships among these capabilities, and the interaction processes of data and control.

In this layer, an experimental procedure is viewed as an execution slice formed by the laboratory system under a specific task context. From this perspective, workflow nodes represent process-level mappings of activated device capabilities, while the relationships between nodes characterize the temporary temporal, control, and data relationships formed among activated resources. Therefore, this paper introduces semantic objects such as interfaces, ports, and lifecycles into workflow nodes. This design shows that a workflow node is not a simple API wrapper, but a process-level capability mapping abstraction of device resources. This abstraction carries execution semantics such as signal transmission and reception, data exchange, and lifecycle evolution.

Formally, the dynamic workflow layer is defined as

$$\mathcal{F} = \langle V_F, C_I, C_P \rangle \quad (7)$$

where V_F denotes the set of workflow nodes, C_I denotes the set of interface connections, and C_P denotes the set of port connections.

The topological structure of the workflow is jointly determined by C_P and C_I . Specifically, The port connection set C_P represents persistent data-dependency channels between nodes. In contrast, the interface connection set C_I specifies the routing relationships for signal interaction, including interaction channels between nodes within the workflow layer and cross-layer interaction channels between nodes and device state machines. The control topology between nodes can be regarded as a subset of the interface connections.

Within this topological structure, each node $n \in V_F$ is defined as

$$n = \langle \psi, X, S, \Theta, I, P, M \rangle \quad (8)$$

where ψ is the node capability, X is the internal variable space, S is the set of node lifecycle states, Θ is the lifecycle transition rules, I is the interface set, P is the port set, and M is the set of internal node actions.

To fully characterize the execution semantics of a workflow node, the detailed definitions and operational mechanisms of each component are elaborated below.

The node capability ψ denotes the semantic function of a node in the workflow. For an execution node, this capability is formulated as $\psi = \langle D, op \rangle$, where D is a device object model and $op \in \Phi$ is one operational capability of that device. For a functional node, ψ denotes control functions such as conditional branching, synchronization, or waiting.

The port set P represents the endpoints through which a node exchanges data with other nodes.

The internal variable space X maintains the local context during node execution, including user-defined variables, port-bound data variables, and attribute data mapped from the device capability model.

The lifecycle state set S describes the process-level execution state of a node, typically including pending, running, completed, and failed. This lifecycle is affected by the state returned from the device state machine, but the two are not equivalent. $\Theta \subseteq S \times S$ defines the allowed lifecycle transition rules. It does not execute actions by itself, but constrains valid transitions between lifecycle states.

The interface set I serves as the endpoints for signal interaction, where each interface $i \in I$ is defined as

$$i = \langle name, dir, type, \Sigma_i, \Gamma_i \rangle \quad (9)$$

where dir denotes the direction, $type$ denotes the interface category, Σ_i defines the set of signals allowed to pass through the interface, and Γ_i denotes the set of triggers bound to the interface. Each trigger is defined as $\gamma = \langle cond, m \rangle$, where $cond$ is a triggering predicate defined over the node context, and $m \in M$ is the node action executed when the

condition is satisfied. When *cond* evaluates to true, the trigger invokes its associated node action. A node action may update the lifecycle state, modify internal variables, or emit signals through interfaces.

During experimental task execution, the dynamic workflow layer operates as follows. After an input interface of a node receives a signal, the triggers bound to that interface check whether their triggering predicates are satisfied. If a condition is satisfied, the corresponding node action is invoked. The node action updates the node context and may request a lifecycle state change. This change must satisfy the lifecycle transition rules specified by Θ . After the context is updated, triggers on other interfaces of the node continue to check whether their triggering predicates are satisfied. If a condition is satisfied, the corresponding action is executed, such as sending a signal through an output interface. Meanwhile, data between nodes remain continuously available through port connections while the connections are valid. Through this mechanism, the workflow can describe the state evolution process and organizational relationships of the laboratory system during task execution, and can coordinate interactions among activated device resources.

3.4. Constraint Layer

The constraint layer C models laboratory requirements and constraints. It describes laboratory-wide global boundary constraints and supplementary constraints introduced during experimental task execution, and monitors whether the current laboratory state remains within admissible boundaries.

Formally, the constraint layer is defined as

$$C = \langle \mathcal{B}_0, \mathcal{O}, \mathcal{H}, \mathcal{U} \rangle \quad (10)$$

where $\mathcal{B}_0$ denotes the set of laboratory-wide global boundary constraints, $\mathcal{O}$ denotes the observable object space, $\mathcal{H}$ denotes the set of violation-handling actions, and $\mathcal{U}$ denotes the set of requirement constraints for experimental tasks.

The observable object space is defined as $\mathcal{O} = \mathcal{O}_R \cup \mathcal{O}_F$, where $\mathcal{O}_R$ denotes the observable objects provided by the static resource layer, including device attributes, command lifecycle states, and device operation states; and $\mathcal{O}_F$ denotes the observable objects provided by the dynamic workflow layer, including node lifecycle states and signal variables.

The set of violation-handling actions $\mathcal{H}$ includes violation-handling actions such as warning, alarm, stop, and abort. These actions are part of the constraint rules and distinguish different types of violation consequences.

$\mathcal{B}_0$ denotes the set of laboratory-wide global boundary constraints that remain continuously active and constrain the operating state of the entire laboratory over time.

Each constraint is defined as

$$\beta = \langle o, \odot, v, h \rangle \quad (11)$$

where $o \in \mathcal{O}$ denotes the observed object, $\odot$ denotes a comparison operator, v denotes a constraint boundary or value range, and $h \in \mathcal{H}$ denotes the handling action associated with the violation of the constraint. For example, $\langle temp, \leq, 200, stop \rangle$ indicates that the observed temperature must not exceed 200. If this condition is violated, the handling action *stop* is executed.

For a task instance τ , the requirement constraint set is defined as $\mathcal{U}_\tau = \langle \mathcal{G}_\tau, \mathcal{B}_\tau \rangle$, where $\mathcal{G}_\tau$ denotes the task goal set, and $\mathcal{B}_\tau$ denotes the boundary constraint set of the experimental task.

In this design, $\mathcal{B}_0$ remains active regardless of whether an experimental task is currently running, whereas $\mathcal{U}_\tau$ is introduced only during the execution of a specific experimental task. Therefore, the constraint layer supports both continuous laboratory-wide supervision and execution assurance for specific experimental tasks.

Specifically, the constraint layer continuously monitors the observable object space during laboratory operation and executes the corresponding handling action when an observed object violates a constraint rule. Device-intrinsic constraints are separated from constraint-layer rules. They are defined by Λ_D in the static resource layer and interpreted locally by the device state machine. For example, when a device temperature exceeds the boundary specified in Λ_D , the state machine immediately detects the violation and enters the corresponding abnormal state. If this abnormal state contains an entry action, such as an alarm, the device state machine triggers that action. At the same time, the abnormal state becomes part of $\mathcal{O}_R$. If the constraint layer defines a corresponding global boundary constraint, such

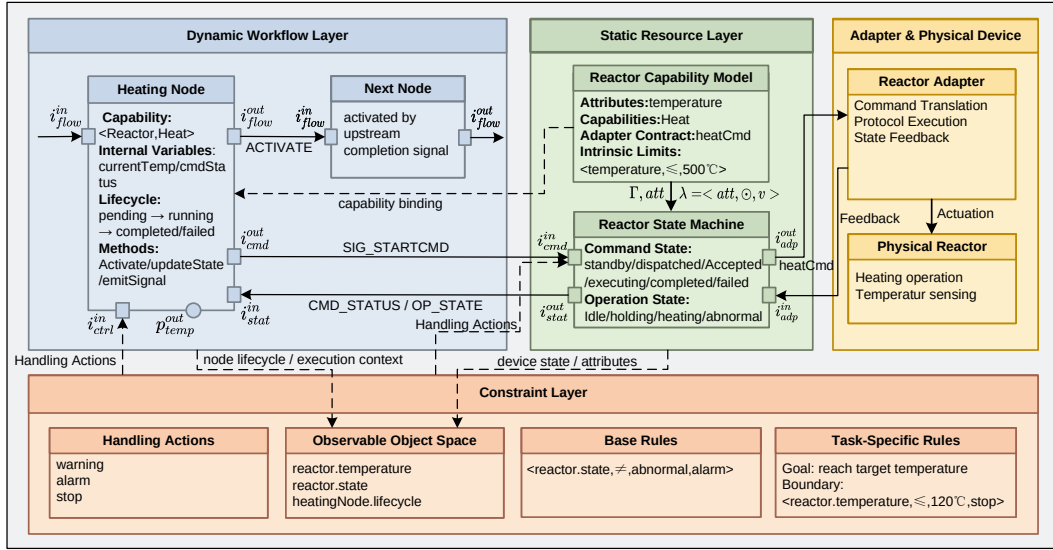


Fig. 1. Instantiated architecture of the proposed modeling framework for the reactor heating task.

as triggering a system alarm when any device becomes abnormal, the constraint layer executes the corresponding system-level handling action based on the observed abnormal state.

The constraint layer can also handle violations that do not belong to device-intrinsic constraints but belong to task constraints. For example, in a given experimental task, the device temperature may not exceed the device-level boundary defined by its own Λ_D , but may exceed the supplementary constraint boundary of that task. In this case, the constraint layer still executes the handling action according to the task constraint in $\mathcal{U}_T$, such as triggering an alarm, and stops the relevant node by sending a *stop* signal to the i_{ctrl}^{in} interface of the workflow node.

Therefore, the constraint layer does not conflict with device-intrinsic constraint handling. Device-intrinsic constraints provide immediate handling of local safety boundaries, while the constraint layer continuously checks observable objects at the global and task levels. Together, they maintain the stable operation of the laboratory system. It should be noted that the constraint layer does not define workflow structure or device behavior. Its mechanism is to continuously perform constraint checking within the current observable space and execute the corresponding handling actions when violations are detected.

4. Results and Discussion

4.1. Case Study

To illustrate how the proposed framework is instantiated and implemented in practical scenarios, this paper adopts a representative reactor heating task as a case study. This case demonstrates three core dimensions of the framework, including device abstraction at the resource layer, device capability mapping at the workflow layer, and runtime supervision at the constraint layer. The objective of this case is not to provide quantitative benchmark tests, but to validate the structural instantiability and runtime executability of the modeling framework.

Figure 1 takes the heating task as an example to demonstrate the instantiated execution architecture of the proposed framework, as well as the specific implementation of the task across the three layers. In the resource layer, the physical reactor is instantiated as an object model with capability definitions and a state machine. In the workflow layer, the heating capability of the object model is mapped to a workflow node, acting as part of the execution slice for the current task. The constraint layer monitors observable objects from the resource layer and the workflow layer. With this architecture, the dynamic evolution of experimental task execution is explicitly represented.

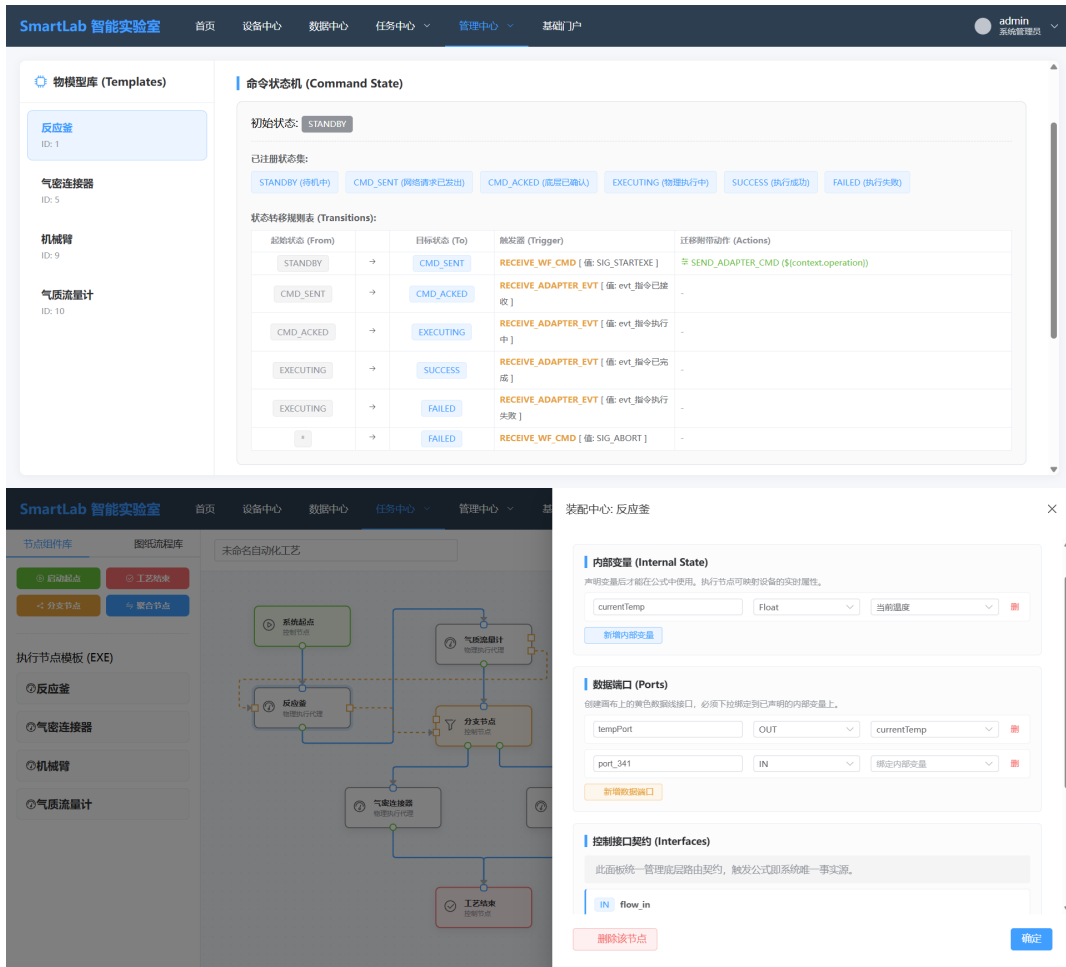


Fig. 2. Prototype system using proposed approach

Figure 2 shows the interface of the developed prototype system. Using a reactor heating workflow as an example, the upper part of the figure presents the state-transition configuration interface of the reactor model state machine, where the states, triggering conditions, and transition actions are defined. The lower part of the figure presents the modeling interface of the workflow layer. When a specific node is selected, relevant information including internal variables, ports and interfaces can be further configured. In this example, the reactor temperature attribute is mapped to the internal variable space of the workflow node and connected to other nodes through data ports, as indicated by the yellow dashed line in the figure. The prototype system shows that the proposed framework can be instantiated and can support the modeling and execution of experimental tasks.

Figure 3 shows the runtime interaction sequence diagram of the case and illustrates the transmission and feedback of execution signals across different layers. The heating task starts when the heating node is activated. The node sends an execution signal to the device state machine. After receiving this signal, the device state machine triggers the corresponding state transition and sends a heating command to the adapter. The adapter parses such commands into physical device protocols and controls the physical reactor to perform heating operations. During execution, the adapter returns feedback events, such as acknowledgment and execution status updates. These events further drive the state evolution of the device state machine, and the updated runtime state is then fed back to the workflow node. After

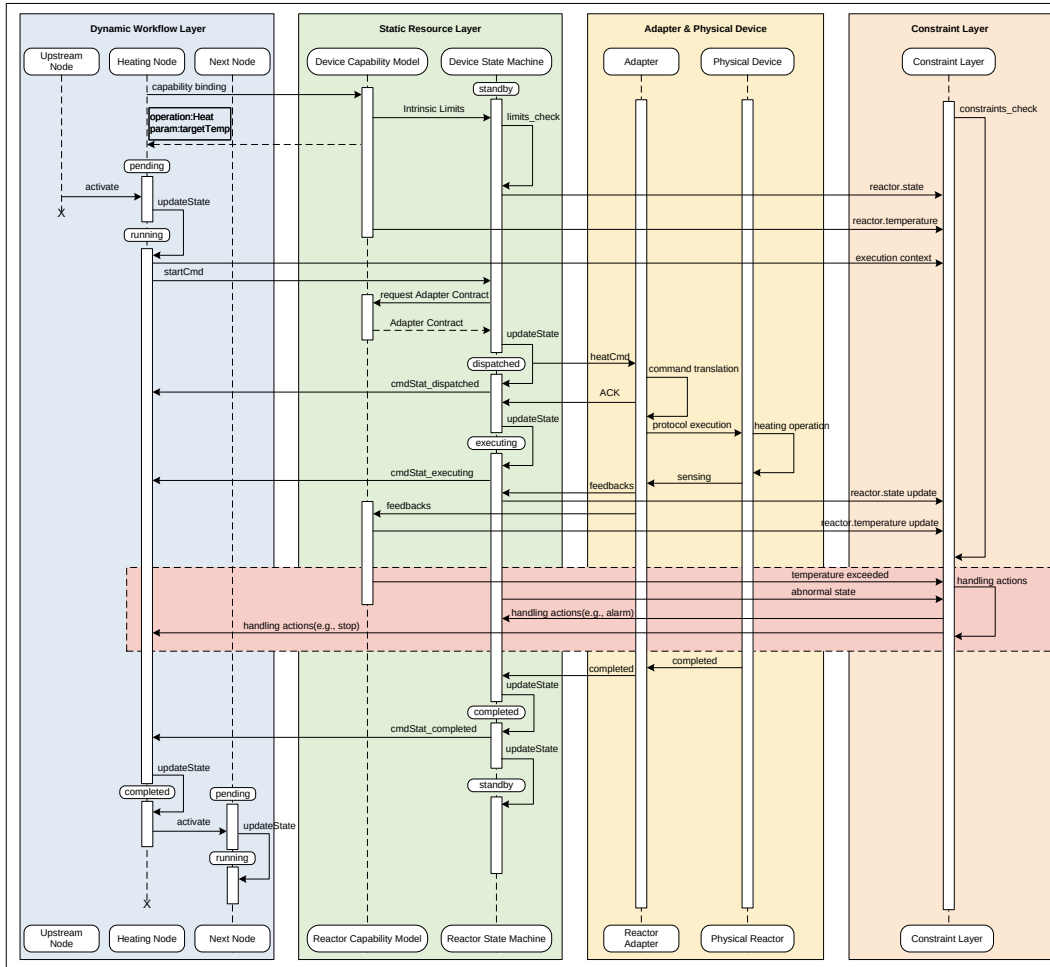


Fig. 3. Cross-layer execution sequence of the reactor heating task.

receiving the completion state, the workflow node updates its lifecycle state and sends the corresponding activation signal to the downstream node.

To further illustrate the role of the constraint layer, an exception branch is introduced in Figure 3. The branch includes two types of exception. The first type occurs when the constraint layer directly detects, in the observable object space, that the reactor temperature exceeds the task constraint boundary. The second type occurs when the device state machine detects that the device temperature exceeds the device-intrinsic limit and transitions into an abnormal state, which is then detected by the constraint layer as an observable state. Both exceptions trigger the corresponding constraint rules and lead to handling actions such as issuing warnings or alarms, or sending stop signals to relevant workflow nodes.

The case study shows that the proposed framework is not just a descriptive abstraction but an executable and instantiable modeling framework. Through this framework, device abstraction, workflow execution, and constraint supervision can be coordinated within a unified model. As a result, the dynamic evolution of experimental task execution is explicitly represented at the model level.

4.2. Discussion

This work focuses on establishing a formal model representation for the execution of experimental tasks. Specifically, a unified modeling framework is developed to characterize the dynamic evolution of laboratory systems throughout task execution from the perspectives of resources, workflows, and constraints.

This modeling approach exhibits prominent advantages. First, device capabilities, equipment states, workflow progression, data dependencies, and constraint rules are no longer hidden within scripts, interfaces and runtime control logic. Instead, they are explicitly defined at the model level, which improves the observability and comprehensibility of the entire experimental execution process. Second, since resources, workflows and constraints are individually modeled and collaboratively integrated within a unified framework, modifications to devices, tasks and constraints can be completed by directly adjusting relevant models. This eliminates the need for tightly coupled code revisions and enhances the configurability and controllability of laboratory systems during experimental operation.

Admittedly, this study still has clear limitations. The present work verifies the executability and representational completeness of the framework mainly through an illustrative case, while systematic validation with more diverse cases and complex task scenarios has not yet been conducted. Therefore, this paper is intended to present a preliminary modeling framework for experimental task execution. Future work will focus on the improvement of prototype systems, the simulated execution of experimental tasks, and systematic verification in more complex experimental scenarios.

Overall, by integrating resources, workflows, and constraints into a unified model architecture, the proposed framework realizes the explicit representation of the dynamic evolution of laboratory systems during the entire task execution process.

5. Conclusions

Experimental task execution in smart laboratories should not be understood merely as the sequential execution of device commands. Instead, it should be regarded as the dynamic evolution of the laboratory system under a specific task context. Based on this view, this paper proposes an executable modeling framework for experimental task execution. The framework represents the laboratory system from three dimensions, namely static resources, dynamic workflows, and execution constraints, and provides an explicit model of the experimental task execution process.

The framework defines workflow as an execution slice formed by the laboratory system under a task context, and represents workflow nodes as process-level abstract mappings of device capabilities. In this way, resources, workflows, and constraints are integrated into a unified model system, so that the dynamic evolution of the laboratory system during task execution can be explicitly represented. The illustrative case further shows that the framework can be instantiated and executed.

Nevertheless, this work still has several limitations. The current study mainly focuses on the proposal of the modeling framework, its formal description, and its illustrative instantiation, but has not yet been systematically validated in richer cases and more complex task scenarios. Future work will focus on prototype system implementation, simulation execution of experimental tasks, and systematic validation in more complex experimental task scenarios.

References

- [1] O. Bayley, E. Savino, A. Slattery, T. Noël, Autonomous chemistry: Navigating self-driving labs in chemical and material sciences, *Matter* 7 (7) (2024) 2382–2398.
- [2] S. D. Rihm, J. Bai, A. Kondinski, S. Mosbach, J. Akroyd, M. Kraft, Transforming research laboratories with connected digital twins, *Nexus* 1 (1) (2024).
- [3] W. Zhang, L. Hao, V. Lai, R. Corkery, J. Jessiman, J. Zhang, J. Liu, Y. Sato, M. Politi, M. E. Reish, et al., Ivoryos: an interoperable web interface for orchestrating python-based self-driving laboratories, *Nature Communications* 16 (1) (2025) 5182.
- [4] F. M. Mione, L. Kaspersetz, M. F. Luna, J. Aizpuru, R. Scholz, M. Borisyak, A. Kemmer, M. T. Schermeyer, E. C. Martinez, P. Neubauer, et al., A workflow management system for reproducible and interoperable high-throughput self-driving experiments, *Computers & Chemical Engineering* 187 (2024) 108720.
- [5] J. Bai, S. Mosbach, C. J. Taylor, D. Karan, K. F. Lee, S. D. Rihm, J. Akroyd, A. A. Lapkin, M. Kraft, A dynamic knowledge graph approach to distributed self-driving laboratories, *Nature Communications* 15 (1) (2024) 462.

- [6] G. Tom, S. P. Schmid, S. G. Baird, Y. Cao, K. Darvish, H. Hao, S. Lo, S. Pablo-García, E. M. Rajaonson, M. Skreta, et al., Self-driving laboratories for chemistry and materials science, *Chemical Reviews* 124 (16) (2024) 9633–9732.
- [7] M. Sim, M. G. Vakili, F. Strieth-Kalthoff, H. Hao, R. J. Hickman, S. Miret, S. Pablo-García, A. Aspuru-Guzik, Chemos 2.0: An orchestration architecture for chemical self-driving laboratories, *Matter* 7 (9) (2024) 2959–2977.
- [8] Y. Fei, B. Rendy, R. Kumar, O. Darts, H. P. Sahasrabudhe, M. J. McDermott, Z. Wang, N. J. Szymanski, L. N. Walters, D. Milsted, et al., Alabos: a python-based reconfigurable workflow management framework for autonomous laboratories, *Digital Discovery* 3 (11) (2024) 2275–2288.
- [9] H. Lee, H. J. Yoo, H. S. Jang, B. Park, Y. J. Park, S. S. Han, Toward self-driving laboratory 2.0 for chemistry and materials discovery, *Materials Horizons* (2026).
- [10] P. Salazar-Villacis, B. Benyahia, The adept framework for assessing autonomous laboratory robotics, *Communications Chemistry* 9 (1) (2026) 99.
- [11] L. Kaspersetz, B. Englert, F. Krah, E. C. Martinez, P. Neubauer, M. N. C. Bournazou, Management of experimental workflows in robotic cultivation platforms, *SLAS technology* 29 (6) (2024) 100214.
- [12] H. Hysmith, E. Foadian, S. P. Padhy, S. V. Kalinin, R. G. Moore, O. S. Ovchinnikova, M. Ahmadi, The future of self-driving laboratories: from human in the loop interactive ai to gamification, *Digital Discovery* 3 (4) (2024) 621–636.
- [13] J. Michael, L. Cleophas, S. Zschaler, T. Clark, B. Combemale, T. Godfrey, D. E. Khelladi, V. Kulkarni, D. Lehner, B. Rumpe, et al., Model-driven engineering for digital twins: opportunities and challenges, *Systems Engineering* 28 (5) (2025) 659–670.
- [14] D. Lehner, J. Zhang, J. Pfeiffer, S. Sint, Model-driven engineering for digital twins: a systematic mapping study: Model-driven engineering for digital twins: a systematic mapping study, *Software and Systems Modeling* (2025).
- [15] S. D. Rihm, Y. R. Tan, W. Ang, M. Hofmeister, X. Deng, M. T. Laksana, H. Y. Quek, J. Bai, L. Pascazio, S. C. Siong, et al., The digital lab manager: Automating research support, *SLAS technology* 29 (3) (2024) 100135.
- [16] S. Lo, S. G. Baird, J. Schrier, B. Blaiszik, N. Carson, I. Foster, A. Aguilar-Granda, S. V. Kalinin, B. Maruyama, M. Politi, et al., Review of low-cost self-driving laboratories in chemistry and materials science: the “frugal twin” concept, *Digital Discovery* 3 (5) (2024) 842–868.
- [17] S. K. Steensen, P. W. Butler, J. R. Pedersen, N. Fisker-Bødker, T. Vegge, J. H. Chang, I. E. Castelli, The necessity of dynamic workflow managers for advancing self-driving labs and optimizers, *Advanced Intelligent Discovery* (2026) 202500067.
- [18] H. Zhang, Y. Li, S. Zhang, L. Song, F. Tao, Artificial intelligence-enhanced digital twin systems engineering towards the industrial metaverse in the era of industry 5.0, *Chinese Journal of Mechanical Engineering* 38 (1) (2025) 40.